

ONNXIM Pipeline / Cycle 对齐可靠性与数值仿真器现状报告

日期：2026-06-22

1. 结论摘要

本报告回答两个问题：

1. 当前 pipeline 时序图和最终 cycle 是否与 C++ 仿真器逻辑可靠对齐。
2. `scripts/general_` 下的通用 Python 数值仿真器距离“可普遍用于验证”还有多远。

核心结论如下：

- 对当前 ONNXIM 代码库而言，最终 `cycle` 结果和基于 trace 绘制的 pipeline 图来自同一套 C++ 执行流程，因此对“解释仿真器自身行为”是可靠的。
- 这张 timeline 不是对真实硬件流水线的逐级还原，但它是对 C++ 仿真器已记录事件的忠实投影。
- 以 `LDLDecompOp` 的 block-LDL 为例，当前时序结果内部是一致的。我重新运行 `example/ldl_test_moderate3.json` 后得到 `Simulation Finished at 3032 cycle`，而导出的 trace 里 `global_max_end` 也是 3032。
- 我已经给 `LDL_block` 补了测试数据和软件求逆 reference。现在可以跑到数值比较阶段，但当前 `LDL_block` 输出与 `inv(H^T H + lambda I)` 软件结果不一致，小尺寸和原始尺寸测试都 FAIL。
- 通用数值仿真器已经不是概念验证阶段。当前的 `InstructionExporter + general_/instruction_sim.py + general_/test_harness.py` 已经形成一条可用的通用验证链路，但目前只覆盖一部分算子/指令。
- AMP-GNN 当前已经可以通过通用 harness 跑通一个 zero-layer 的 demo；但旧的 `verify_amp_gnn.py` 仍然是 operator-specific 的专用验证脚本，不能视为最终通用方案。

2. Pipeline / Cycle 对齐可靠性

2.1 最终 cycle 来自哪里

最终 cycle 不是 Python 后处理算出来的，而是 C++ 仿真主循环直接跑出来的。

在 `src/Simulator.cc` 中：

- `Simulator::cycle()` 执行 `while (running())`。
- 每次 core 时钟域触发时，会调度工作、推进每个 core 的 `_cores[core_id]->cycle()`，然后将 `_core_cycles++`。
- 仿真结束后打印 `Simulation Finished at {} cycle`。

因此，最终报告的 cycle 就是 C++ 仿真器内部维护的全局 core cycle 计数器。

2.2 timeline trace 来自哪里

timeline CSV 同样来自 C++ 仿真器，是在执行边界上显式打点得到的。

在 `src/Core.cc` 中：

- `push_memory_response()` 在内存响应回到 core 时记录 Load / Store，对应 CoreX_MTE2 / CoreX_MTE3。

在 `src/Core.cc` 中：

- `finish_compute_pipeline()` 记录 CoreX_Cube
- `finish_vector_pipeline()` 记录 CoreX_Vector
- `finish_scalar_pipeline()` 记录 CoreX_Scalar

仿真结束后，`src/Simulator.cc` 会检查 `ONNXIM_TRACE_CSV`，然后通过 `TraceLogger::dump_to_csv(...)` 导出 CSV。

所以三者关系是：

- `cycle`：来自 C++ 仿真主循环
- `trace csv`：来自 C++ 执行过程中的事件打点
- `png`：读取 CSV 后绘图得到

2.3 可视化脚本做了什么、没做什么

可视化脚本不会重新计算仿真时序，它只是读取 CSV 并画图。

在 `visualizer_png.py` 中：

- 读取 CSV
- 规范化列名
- 按 core / unit 分组
- 绘制 Cube、Vector、MTE2、MTE3、CubeWait

需要注意的一点：

- 如果 trace 里没有显式的 CubeWait，脚本可以根据相邻 Cube 事件之间的空隙推断一个“等待区间”并画成灰色（`visualizer_png.py`）。
- 因此 PNG 主要是对 trace 事件的直接可视化，但确实包含少量仅用于显示的 wait 推断逻辑。

这不会影响最终 cycle，只会影响图上空白 cube gap 的显示方式。

2.4 这里说的“可靠对齐”是什么意思

这里合理的判断标准应该是：

- pipeline 图中的 bar 是否来自同一次仿真执行；
- trace 的 `max(end_cycle)` 是否和最终 simulator 的结束 cycle 一致；
- operator lowering 中的主要算法阶段，是否能在 trace 中按预期顺序出现。

对当前 ONNXIM 来说，这三个问题的答案都是“是”，因此它对于“分析仿真器内部行为”是可靠的。

但这并不意味着 PNG 是真实芯片内部逐级流水线波形。它的含义是：这张图能够可靠地概括当前 C++ 仿真器的执行模型。

3. 以 LDL block 为例说明

3.1 为什么选 LDL block

`LDLDecompOp` 是一个很好的例子，因为它同时包含：

- MTE load/store
- Cube 计算
- Vector / Scalar 计算
- 多个显式 barrier
- 多阶段算法流程

因此它比单一 GEMM 更能覆盖仿真器的关键路径。

3.2 当前 C++ lowering 实际做了什么

在 `src/operations/LDLDecompOp.cc` 中，block-LDL 大致被 lowering 成以下步骤：

1. 将 `H` 和 regularizer 矩阵 `RegI` 搬入 SPAD。
2. load 到 Gram 之间加 barrier。
3. 计算 Gram: $G = H^H H$ 。
4. Gram 到 regularization 之间加 barrier。
5. 计算 $A = G + \text{lambda } I$ 。
6. 进入 block-LDL 主体前加 barrier。
7. 对每个 block:
 - 更新 `D_jj`
 - 做对角块倒数/逆相关步骤
 - 做 `L` 的块更新
 - 每个阶段之间插 barrier
8. 执行 backward-substitution 风格的逆矩阵装配。
9. store 之前加 barrier。
10. `MOVOUT` 最终输出逆矩阵。

这类 staged algorithm 在 trace 中理应能看到明确的阶段结构。

3.3 重新实跑结果

我重新运行了：

```
ONNXIM_HOME=/workspace \  
ONNXIM_TRACE_CSV=tmp/ldl_moderate3_current_trace.csv \  
ONNXIM_MAX_CORE_CYCLES=10000000 \  
./build/bin/Simulator \  
  --config configs/ascend_910b_quiet.json \  
  --models_list example/ldl_test_moderate3.json \  
  --mode ldl_test \  
  --log_level info
```

从 `tmp/ldl_moderate3_current.log` 观察到：

- Layer finish: 3027

- Simulation finished: 3032 cycle
- Core0 total cycle: 3032

从 tmp/ldl_moderate3_current_trace.csv 统计得到:

- global_max_end = 3032
- core0_max_end = 2962
- span = 3031

这组数据是严格对齐的:

- simulator 最终结束 cycle = 3032
- trace 中最大 end_cycle = 3032

并且它和历史报告 results/LDL/falsification/LDL_BLOCK_MODERATE3_COMPARE_20260327.md 中记录的结果一致:

- 全局周期 3032
- Core0 结束周期 2962
- global_max_end = 3032

说明这不是一次偶然对齐, 而是该 operator 当前模型下稳定成立的结果。

3.4 这个 LDL block 结果是否合理

是合理的, 前提是我们讨论的是“在当前仿真器抽象下是否合理”。

原因如下:

- 代码中的算法阶段顺序, 与 trace 中应出现的阶段顺序一致。
- trace 的事件组成是合理的。现有 ldl_block_64x16_trace_moderate3.csv 的统计为:
Vector=25632, MTE2=3840, Cube=1248, Wait=1084, MTE3=768
- 这说明当前 block-LDL 并不是一个纯 GEMM 密集型 kernel, 而是包含大量块更新、倒数/乘法、ADD/MUL、同步控制等工作。
- Layer finish at 3027 与 Simulation Finished at 3032 只差少量 cycle, 也符合预期: layer 本身可能已经完成, 但 simulator 还要再 drain 一点残余状态/队列。

3.5 对 LDL block 的可靠性判断

对 LDLDecompOp block 模式来说, 当前 timing 结果可以认为是对 C++ 仿真器自身行为的可靠描述:

- operator lowering
- cycle 结果
- pipeline 图

三者是相互一致的。

真正需要保留的 caution 不是“内部不一致”, 而是“抽象层级”:

- Load 条表示的是从 request 创建到 response 被 core 消费的端到端区间, 并不是现实硬件里 DMA issue / NoC / DRAM / return path 的细粒度拆分。
- 因此这张 timeline 是可靠的 simulator trace, 而不是芯片级物理流水线波形。

4. 数值仿真器当前进度

4.1 现在已经具备什么

当前通用链路已经形成三层结构：

1. C++ lowering / export: `src/InstructionExporter.cc` 负责导出 `schema`、`config`、`tensors`、可选 `verification` 和 flat `instructions`。
2. 通用硬件风格 replay: `scripts/general_/instruction_sim.py` 只执行 flat DRAM / SPAD / ACCUM 和记忆体上的 generic opcode。
3. tensor-aware verification harness: `scripts/general_/test_harness.py` 负责读取 `verification`、初始化 DRAM、用 NumPy 计算 reference、做数值比较。

这是正确的总体架构。

4.2 通用模拟器当前已支持的能力

在 `scripts/general_/instruction_sim.py` 中，当前 generic simulator 已支持：

- `MOVIN`
- `MOVOUT / MOVOUT_POOL`
- `GEMM / GEMM_PRELOAD / GEMM_WRITE`
- `RMSNORM`
- 向量/标量类指令: `ADD`、`MUL`、`MAC`、`DIV`、`ADDTREE`、`EXP`、`SQRT`、`COMP`、`SOFTMAX`、`GELU`、`SWISH`、`SCALAR_*`
- 结构型指令: `BAR`、`PIPE_BARRIER`

这已经足以支撑一部分真实导出程序的数值验证。

4.3 距离“普遍可用”还差什么

如果说“generic numeric simulator 已经可以普遍验证大多数 ONNXIM operator”，现在还不能这么说。

主要差距有：

- opcode 覆盖还不全：例如 `LAYERNORM` 在 C++ export 中存在，但在 `scripts/general_/instruction_sim.py` 的 dispatch 中还没有接上。`IM2COL` 也不在当前 generic simulator 的支持范围内。
- verification / reference 表达能力还有限：这里的 reference 指的是 harness 验证层的“标准答案”，不是 Python 硬件指令仿真器本身。在 `scripts/general_/test_harness.py` 中，reference DSL 目前只支持：`matmul`、`add`、`sub`、`mul`、`transpose`、`reshape`、`softmax`、`amp_gnn_zero_layer` 还没有通用的 LDL / Cholesky / solver 类 reference op。
- 模型接入覆盖不足：harness 要求 JSON 中带 `verification` 段 (`scripts/general_/test_harness.py`)。很多旧导出 JSON 没有这部分内容，会直接失败。这意味着目前只有实现了 `get_verification_spec()` 的模型，才能真正接入这条通用路径。
- 复杂 lowering 还未完全 generic 化：一些工作负载在变成“真正可通用验证”的 generic 指令流之前，还需要先补 reusable lowering primitive。

4.4 LDL_block 数学操作与软件求逆对比

LDL_block 在当前模型中的输入/输出很明确：

- 输入 **H**：形状为 $[B, M, U]$
- 输入 **RegI**：形状为 $[U, U]$
- 输出 **Ainv**：形状为 $[B, U, U]$

因此它并不是完整的信号检测算法。完整线性检测一般还需要 **y**，并计算类似：

$$\mathbf{x_hat} = \text{inv}(\mathbf{H}^T \mathbf{H} + \text{lambda } \mathbf{I}) \mathbf{H}^T \mathbf{y}$$

但当前 LDL_block 没有 **y** 输入，也不输出 **x_hat**。它覆盖的是其中的矩阵预处理/求逆部分：

$$\begin{aligned} \mathbf{G} &= \mathbf{H}^T \mathbf{H} \\ \mathbf{A} &= \mathbf{G} + \text{RegI} \\ \mathbf{Ainv} &= \mathbf{A}^{-1} \end{aligned}$$

在 C++ lowering 中，代码阶段大致对应：

1. 搬入 **H** 和 **RegI**： [src/operations/LDLDecompOp.cc](#)
2. 计算 Gram 矩阵，注释语义为 $\mathbf{G} = \mathbf{H}^T \mathbf{H}$ **H**： [src/operations/LDLDecompOp.cc](#)
3. 做 regularization，注释语义为 $\mathbf{A} = \mathbf{G} + \text{lambda } \mathbf{I}$ ： [src/operations/LDLDecompOp.cc](#)
4. 进入 block-LDL inspired 的 D/L 更新和 backward substitution： [src/operations/LDLDecompOp.cc](#)
5. 最后把 **Ainv** 写回 DRAM： [src/operations/LDLDecompOp.cc](#)

为了测试“求逆是否正确”，我在 verification 层使用 NumPy 软件 reference：

$$\text{expected} = \text{inv}(\mathbf{H}^T \mathbf{H} + \text{RegI})$$

这个 reference 不参与硬件 replay，只作为标准答案。当前已经补充：

- LDLModel::get_verification_spec() 导出 **H**、**RegI**、**Ainv** 的验证信息： [src/models/LDLModel.cc](#)
- scaled_identity initializer 生成 $\text{RegI} = \text{lambda } \mathbf{I}$ ： [scripts/general/test_harness.py](#)
- 软件求逆 reference 在 harness 中计算： [scripts/general/test_harness.py](#)
- 小尺寸测试配置用于观察矩阵结果： [example/ldl_test_inverse_small.json](#)

小尺寸求逆测试

小尺寸配置为 **M=8**、**U=4**、**B=1**。导出后：

- Simulation Finished at 168 cycle
- instruction 数量：34
- has_verification = True

运行：

```
python3 scripts/general_/run_harness.py tmp/ldl_inverse_small_export.json
```

结果为 **FAIL**:

```
[FAIL] ldl_inverse_small.Ainv shape=[1, 4, 4]
max_abs_err=3.160499e+00
rel_err=1.000000e+00
mean_abs_err=7.197024e-01
rmse=1.136422e+00
```

当前 **LDL_block** 输出为:

```
[[[3.06363794e+00 0.00000000e+00 1.91941195e-02 2.98308994e-03]
 [0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
 [0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
 [0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]]]
```

软件求逆结果为:

```
[[[ 1.941764 -0.1158785 -0.0669005 -0.84772762]
 [-0.1158785 1.73868991 0.22319351 -0.16985201]
 [-0.0669005 0.22319351 2.15500781 0.2349436 ]
 [-0.84772762 -0.16985201 0.2349436 3.16049856]]]
```

原始尺寸求逆测试

对原来的 **example/ldl_test_moderate3.json** 重新导出后:

- **Simulation Finished at 3032 cycle**
- instruction 数量: **27168**
- **has_verification = True**

运行:

```
python3 scripts/general_/run_harness.py tmp/ldl_moderate3_inverse_export.json
```

结果同样为 **FAIL**:

```
[FAIL] ldl_64x16_moderate3.Ainv shape=[96, 16, 16]
max_abs_err=7.495186e+02
rel_err=8.205838e+02
```

```
mean_abs_err=3.294105e-01
rmse=5.066717e+00
```

为什么当前求逆不正确

当前结果不正确的核心原因不是 verification 层，而是 `LDL_block` 的 C++ lowering 还没有完整表达真正的矩阵求逆算法。具体来说：

1. $G = H^T H$ 的转置语义没有真正表达出来。

数学上，如果 H 是 $[M, U]$ ，那么 $H^T H$ 的第 (i, j) 个元素应该是：

```
G[i, j] = sum_m H[m, i] * H[m, j]
```

也就是说，左边输入需要按 H 的列来读。例如小测试里 H 是 $[8, 4]$ ，计算 $G[0, 0]$ 时应该读：

```
H[0,0], H[1,0], H[2,0], ..., H[7,0]
```

这些元素在 row-major 内存里不是连续的，它们之间隔着 $U=4$ 个元素。可是当前 `LDL_GRAM` 的指令是用同一个 `addr_H` 作为两个 GEMM 输入：

```
src_addrs = {addr_H, addr_H}
tile_m = U
tile_k = M
tile_n = U
```

这里没有导出“第一个输入按转置视图读取”的信息，也没有 2D stride 能表达 `row_stride=1, col_stride=U` 这种访问方式。因此 Python 数值 replay 很难把这条 GEMM 正确解释成 $H^T H$ 。它更像是在对同一片连续内存做一个普通 GEMM 形状读取，而不是严格的 Gram 矩阵计算。

2. block-LDL 的 D/L 更新没有完整表达 Schur complement。

标准 LDL 分解要满足：

```
A = L D L^T
```

如果按 block 做，以两个 block 为例：

```
A = [ A00  A01
      A10  A11 ]
```


正确流程大致应该是：

```
D0 = A00
L10 = A10 * inv(D0)
D1 = A11 - L10 * D0 * L10^T
```

关键点是 **D1** 不是直接从 **A11** 来的，也不是简单乘一下 **A**；它要减去前面 block 已经贡献过的 Schur complement。当前代码中的 **LDL_D_UPDATE_j** 使用的是：

```
src_addrs = {addr_A, addr_A}
```

这更像是用 **A** 和 **A** 做一个计算负载 pattern。它没有明确读取 **A_jj**、**L_jk**、**D_k**，也没有表达 **A_jj - sum(...)** 这个减法更新。因此它不能还原标准 block-LDL 的数学递推。

3. 多个 block 没有明确的 block offset 读写语义。

对 **U=4**，**block_size=2** 的小例子，理论上矩阵会被分成 4 个 **2x2** block：

```
A00: rows 0..1, cols 0..1
A01: rows 0..1, cols 2..3
A10: rows 2..3, cols 0..1
A11: rows 2..3, cols 2..3
```

如果要处理 **A11**，地址应该偏移到：

```
addr_A + (2 * U + 2) * precision
```

但当前很多 block 指令仍然使用同一个 **addr_A**、**addr_Dinv**、**addr_tmp**、**addr_Ainv** 基址，没有把 **(block_row, block_col)** 显式编码进 **src_addrs** 或 **dest_addr**。这会导致不同 block 的计算在同一片 scratch / accumulator 区域上覆盖或重复计算。

小尺寸测试的实际输出也能看到这种现象：**Ainv** 只有第一行少量位置非零，其余大部分为 0，而软件求逆结果是一个完整的 dense **4x4** 矩阵。这说明当前 microprogram 没有把完整 **U x U** 逆矩阵的所有 block 正确写出来。

4. **SCALAR_DIV** 不能等价于对角块矩阵求逆。

如果 block size 是 **2**，一个对角块可能是：

```
D = [ a  b
      c  d ]
```

这个 2×2 矩阵的逆不是逐元素倒数，而是：

$$\text{inv}(D) = 1 / (a*d - b*c) * \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

也就是说，要求 determinant，还要交换对角元素、改变非对角符号。当前 `LDL_D_DIAG_INV_j` 使用的是：

```
opcode = SCALAR_DIV
src_addrs = {addr_Reg, addr_Ainv}
compute_size = block_size * block_size
```

这是一条逐元素除法路径。逐元素除法最多能表达类似 $x[i] / y[i]$ ，不能表达 2×2 或更大 block 的矩阵逆。除非 block size 固定为 1，且公式退化成标量倒数，否则这一步不能作为真正的 D_{jj}^{-1} 。

5. backward substitution 没有完整还原 $A^{-1} = L^{-T} D^{-1} L^{-1}$ 。

标准 LDL 求逆不是只算出 D^{-1} 就结束。还需要解两个三角系统：

$$A^{-1} = L^{-T} D^{-1} L^{-1}$$

这意味着要有明确的 L block 存储，并按列或按 block 做依赖正确的 forward/backward solve。比如要求某一列的结果时，后面的 off-diagonal block 会依赖之前已经解出的列。

当前代码的 backward 阶段确实制造了从后往前的依赖和若干 MAC/ADD：

```
LDL_BWD_DIAG_MUL
LDL_BWD_DIAG_ACC
LDL_BWD_OFF_MUL
LDL_BWD_OFF_ACC
```

但这些指令主要还是读 `addr_Dinv` 和 `addr_A`，没有一个完整的 L 矩阵存储/读取路径，也没有把每个 block 的 triangular solve 关系完整表达出来。因此它可以形成合理的 timing 负载和依赖链，但数值上不能保证得到 $L^{-T} D^{-1} L^{-1}$ 。

因此现在的结论是：

- `LDL_block` 的 timing / pipeline 行为仍然是可靠的 C++ 仿真器 trace。
- `LDL_block` 从接口和命名上看是在做 `Ainv`，也就是检测链路里的求逆/预处理部分。
- 但从数值验证看，当前 `Ainv` 不等于软件计算的 $\text{inv}(H^T H + \text{RegI})$ 。
- 所以它还不能作为“求逆正确”的数值算子使用。

4.5 AMP-GNN 当前的真实状态

AMP-GNN 是目前最有代表性的进展标记。

当前 generic 路径

[src/models/AmpGNNModel.cc](#) 现在已经能够导出标准 **verification**:

- tensor metadata
- input initializer
- output expectation
- comparison tolerance

其中 **expected** 当前使用的是 **amp_gnn_zero_layer**, generic harness 已经能用 NumPy 计算这个 reference ([scripts/general/test_harness.py](#))。

我重新导出当前 AMP-GNN JSON 后, 直接运行:

```
python3 scripts/general/run_harness.py tmp/amp_gnn_current_export.json
```

结果为:

- **PASS**
- model: **amp_gnn_test_model**
- instruction 数量: 96
- 输出张量: **amp_gnn_test_model.AMP_GNN.x_hat**
- 当前运行下误差为 0

这说明 **通用 harness 路径已经真实可用**, 不是纸面设计。

但边界也很明确

当前 generic AMP-GNN lowering 只支持 **gnn_layers = 0**。

在 [src/operations/AmpGNNWorkload.cc](#) 中:

- 如果 **gnn_layers != 0**, 会直接抛异常
- 注释明确说明: message-passing 部分还缺 reusable gather/broadcast lowering, 因此还不能放到 generic instruction simulator 路径中验证

所以要严格区分:

- AMP-GNN zero-layer readout path: 已经能走通 generic path
- 完整 message-passing AMP-GNN: 还没有走通 generic path

4.6 **verify_amp_gnn.py** 是否通用

不是。

[scripts/verify_amp_gnn.py](#) 是一个明显的 operator-specific replay 脚本, 它:

- 按 AMP-GNN 专属 **inst_id** 分支

- 重建 AMP 迭代状态机
- 硬编码 MPNN / AMP-GNN 语义
- 要求 JSON 中存在 AMP-GNN metadata ([scripts/verify_amp_gnn.py](#))

它适合作为调试工具，但不能算通用数值仿真器。

而且我把它跑在当前重新导出的 generic JSON 上时，它直接报：

- No AMP-GNN metadata found in instruction JSON

这个失败本身反而说明项目方向是对的：

- 说明现在系统正从 operator-specific replay metadata
- 迁移到 [verification + instruction_sim.py + test_harness.py](#)

这才是通用验证的正确长期方向。

4.7 距离“可普遍用于验证”还有多远

当前距离不算远，但还不能说已经完全就绪。

已经完成的部分：

- flat ISA-like export 已经有了
- generic replay engine 已经有了
- tensor-aware harness 已经有了
- 至少有一个非平凡 demo (AMP-GNN zero-layer) 已经端到端 PASS

还需要补的部分：

1. 在 [scripts/general_/instruction_sim.py](#) 中补齐缺失 opcode 的执行语义。
2. 在 harness 的 verification / reference DSL 中补更多可复用 reference op，尤其是 LDL / Cholesky / solver 类。
3. 让更多 C++ model/operator 实现 [get_verification_spec\(\)](#)。
4. 继续把 operator-specific 验证逻辑从最终主路径中移除。
5. 建立一套稳定的 exported JSON regression 集合和 PASS 回归。

5. 最终判断

5.1 关于 pipeline / cycle 可靠性

当前 pipeline 图和最终 cycle 对 **ONNXIM C++ 仿真器自身逻辑** 来说是可靠对齐的。

原因是：

- 两者来自同一次仿真执行；
- trace 来自 C++ 显式事件打点；
- visualizer 是 CSV 渲染器，而不是第二个仿真器。

[LDLDecompOp](#) block 例子进一步证明了这一点：

- simulator final cycle: 3032
- trace [global_max_end](#): 3032

- 历史归档报告与当前复跑结果一致

5.2 关于数值仿真器进度

通用数值仿真器已经不是“还在设计中”，而是“部分可用”。

最重要的现实证据是：

- 通用 harness 已经可以对当前 AMP-GNN zero-layer export 做端到端验证并 PASS
- LDL_block 已经补上 verification 和软件求逆 reference；当前能跑到数值比较阶段，但结果 FAIL，说明 LDL_block 当前数值 lowering 尚未等价于 $\text{inv}(H^T H + \lambda I)$
- 旧的 `verify_amp_gnn.py` 则明确不是通用方案，应被视为专项调试工具

因此当前最准确的状态描述是：

- **pipeline / cycle 对齐用于仿真器分析：现在可用**
- **generic instruction-level numeric verification：现在部分可用**
- **full operator coverage / full complex-model coverage：仍未完成**